	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.C. René Adrián Dávila Pérez

Asignatura: Programación orientada a objetos

Grupo: 07

No de práctica(s): 11

Integrante(s): 424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada: 1

Semestre: 2027-1

Fecha de entrega: 13 de noviembre de 2026

Observaciones:

Calificación:

ÍNDICE

1. INTRODUCCIÓN

1.1. Planteamiento del Problema

1.2. Motivación

1.3. Objetivos

1.3.1. Objetivo General

1.3.2. Objetivos Específicos

2. MARCO TEÓRICO

3. DESARROLLO

3.1. Descripción de la Implementación

3.2. Consultas de GUI con IA Generativa por la Brigada

3.3. Diagramas UML

3.4. Pruebas de Escritorio

4. RESULTADOS

5. CONCLUSIONES

6. REFERENCIAS

1. INTRODUCCIÓN

1.1. Planteamiento del Problema

En el desarrollo de sistemas orientados a objetos, el almacenamiento volátil en la memoria principal resulta insuficiente para la persistencia de información y el procesamiento de volúmenes de datos estructurados. Las aplicaciones requieren interactuar de forma robusta con fuentes externas, específicamente archivos de texto plano con extensión .txt, para extraer su contenido, manipularlo lógicamente y presentar los resultados al usuario a través de componentes gráficos especializados. El problema planteado en esta práctica consiste en diseñar e implementar una arquitectura modular capaz de leer un archivo externo, procesar léxicamente su contenido para calcular el recuento total de palabras y mostrar tanto el texto íntegro como la métrica obtenida dentro de un cuadro de texto en una interfaz gráfica de usuario (GUI), integrando tecnologías de ventanas en Java.

1.2. Motivación

La capacidad de leer y manipular archivos externos constituye un pilar indispensable en la ingeniería de software moderna para el intercambio de datos y la persistencia de configuraciones. Abstraer las operaciones de entrada y salida mediante flujos de datos (streams) bajo el paradigma orientado a objetos permite desacoplar la lógica de negocio de los dispositivos de

almacenamiento. Asimismo, vincular el procesamiento de archivos con componentes de interfaz gráfica potencia la experiencia del usuario, transformando datos planos en representaciones visuales claras y funcionales que facilitan la auditoría y consulta de la información procesada.

1.3. Objetivos

1.3.1. Objetivo General

Implementar el intercambio de datos (lectura y escritura) entre fuentes externas (archivos y/o entrada y salida estándar) y un programa (en un lenguaje orientado a objetos).

1.3.2. Objetivos Específicos

- Investigar y estructurar el uso de componentes de cuadros de texto en interfaces gráficas de usuario en Java, abarcando tecnologías como Swing y JavaFX mediante la consulta asistida por herramientas de Inteligencia Artificial Generativa (Ejercicio 1).
- Implementar el algoritmo de lectura y procesamiento de archivos con extensión .txt para extraer el contenido textual y calcular el número total de palabras (Ejercicio 2).
- Diseñar la arquitectura del software bajo el principio de separación de responsabilidades, aislando la lógica de lectura y conteo (LectorArchivo) de la gestión visual y el contenedor gráfico (VentanaContador).
- Validar el funcionamiento del sistema mediante pruebas experimentales de ejecución gráfica, comprobando la correcta visualización del archivo de texto y el despliegue exacto del recuento de palabras en la ventana.

2. MARCO TEÓRICO

El manejo de archivos en la Programación Orientada a Objetos se fundamenta en la abstracción de flujos de entrada y salida (input/output streams), los cuales permiten la transferencia secuencial de bytes o caracteres entre una fuente externa y la memoria de la aplicación **[1]**. En la plataforma Java, el paquete java.io provee las clases esenciales para la lectura de archivos de texto plano mediante clases optimizadas que operan con búferes de memoria (BufferedReader y FileReader), reduciendo el número de accesos directos al disco y maximizando la eficiencia en el procesamiento de cadenas **[2][3]**.

Para garantizar que el flujo de ejecución no colapse ante incidencias físicas o lógicas, las operaciones de E/S exigen la gestión obligatoria de excepciones verificadas, tales como IOException, las cuales interceptan fallos por rutas inexistentes o permisos denegados **[1][3]**.

Por otro lado, la construcción de interfaces gráficas de usuario (GUI) en Java se realiza mediante librerías especializadas **[4]**. En Swing, componentes como JTextArea encapsulan áreas de texto multilínea dotadas de capacidades de desplazamiento y edición, mientras que en JavaFX se emplean componentes equivalentes vinculados al motor de renderizado gráfico de la plataforma **[4][5]**. La integración de archivos de texto con una GUI exige una arquitectura orientada a objetos donde una clase gestora procese el archivo y suministre los datos limpios hacia los componentes visuales de la ventana principal **[6]**.

3. DESARROLLO

3.1. Descripción de la Implementación

El sistema se diseñó bajo una arquitectura modular compuesta por tres clases principales:

1. **Clase Principal (Main):** Contiene el método estático main(String[] args), encargado de inicializar el hilo de ejecución principal y desplegar la ventana gráfica del sistema.
2. **Clase de Interfaz Gráfica (VentanaContador):**
    - Modela el contenedor visual de la aplicación configurando sus dimensiones espaciales y disponiendo los elementos gráficos necesarios para cumplir con el Ejercicio 1 (cuadros de texto en GUI).
    - Contiene atributos que referencian a un objeto lector de archivos (lector: LectorArchivo), un componente de área de texto multilínea para mostrar el contenido del documento (areaTexto: JTextArea) y una etiqueta inferior para desplegar el resultado métrico (etiquetaPalabras: JLabel).
    - Gestiona el flujo visual invocando los métodos de extracción de datos y actualizando las propiedades de texto de los componentes gráficos.
3. **Clase de Procesamiento y Lectura (LectorArchivo):**
    - Da cumplimiento directo al Ejercicio 2 al encapsular los atributos de contenido textual (Contenido: String) y el acumulador numérico de palabras (totalPalabras: int).
    - Implementa el método leer(File archivo), el cual abre el flujo de lectura secuencial del archivo .txt, procesa las líneas del documento y aplica operaciones de segmentación léxica mediante patrones de separación de espacios para cuantificar con precisión el total de palabras contenidas.
    - Provee métodos consultores (getContenido() y getTotalPalabras()) que entregan la información procesada hacia la interfaz gráfica para su renderizado en pantalla.

3.2. Consultas de GUI con IA Generativa

De acuerdo con las indicaciones del profesor, los cinco integrantes de la brigada consultamos herramientas de Inteligencia Artificial Generativa distintas para investigar el uso de cuadros de texto en GUI:

- **Integrante 1 (ChatGPT - OpenAI):**
    - Prompt: "¿Cómo se implementa un cuadro de texto multilínea en Java Swing utilizando JTextArea y cómo se integra con barras de desplazamiento?"
    - Respuesta de la IA: Explicó que JTextArea permite mostrar texto extenso, pero requiere ser envuelto dentro de un contenedor JScrollPane para habilitar barras de desplazamiento automáticas cuando el contenido rebasa las dimensiones visuales de la ventana.

- **Integrante 2 (Claude - Anthropic):**
    - Prompt: "Explica la arquitectura y el uso de componentes de texto (TextArea y TextField) en JavaFX para aplicaciones de escritorio."
    - Respuesta de la IA: Detalló que JavaFX maneja nodos de control de texto altamente personalizables mediante hojas de estilo CSS y enlace de propiedades observables (*properties*), facilitando la actualización dinámica de datos leídos desde archivos.
- **Integrante 3 (Gemini - Google):**
    - Prompt: "¿Qué consideraciones de hilos de ejecución (Event Dispatch Thread) se deben tener al actualizar un cuadro de texto en una GUI tras leer un archivo grande?"
    - Respuesta de la IA: Señaló que todas las modificaciones a los componentes visuales de la interfaz deben ejecutarse en el hilo dedicado de despacho de eventos de la GUI para evitar bloqueos e inconsistencias de renderizado en la ventana.
- **Integrante 4 (Copilot - Microsoft):**
    - Prompt: "¿Cómo configurar el ajuste de línea automático (line wrap) en un cuadro de texto gráfico para evitar que las oraciones largas se desborden horizontalmente?"
    - Respuesta de la IA: Indicó que mediante la invocación del método de ajuste de línea y palabras en el componente de texto se logra que el contenido se adapte al ancho del contenedor gráfico de manera limpia.
- **Integrante 5 (DeepSeek):**
    - Prompt: "¿Cuáles son las mejores prácticas para manejar excepciones de E/S y reflejar mensajes de error directamente en un cuadro de texto gráfico?"
    - Respuesta de la IA: Recomendó capturar la excepción en la capa lógica y utilizar el método de asignación de texto del cuadro gráfico (setText()) para mostrar un reporte descriptivo de la falla al usuario sin interrumpir la ejecución de la ventana.

3.3. Diagramas UML

A la par del diseño modular, el sistema fue modelado mediante diagramas formales en el lenguaje unificado de modelado:

Diagrama de Clases UML: Representa la estructura estática del sistema, evidenciando la clase de arranque Main, su dependencia con VentanaContador y la asociación bidireccional de esta última con la clase de procesamiento LectorArchivo, detallando sus atributos y métodos públicos y protegidos.

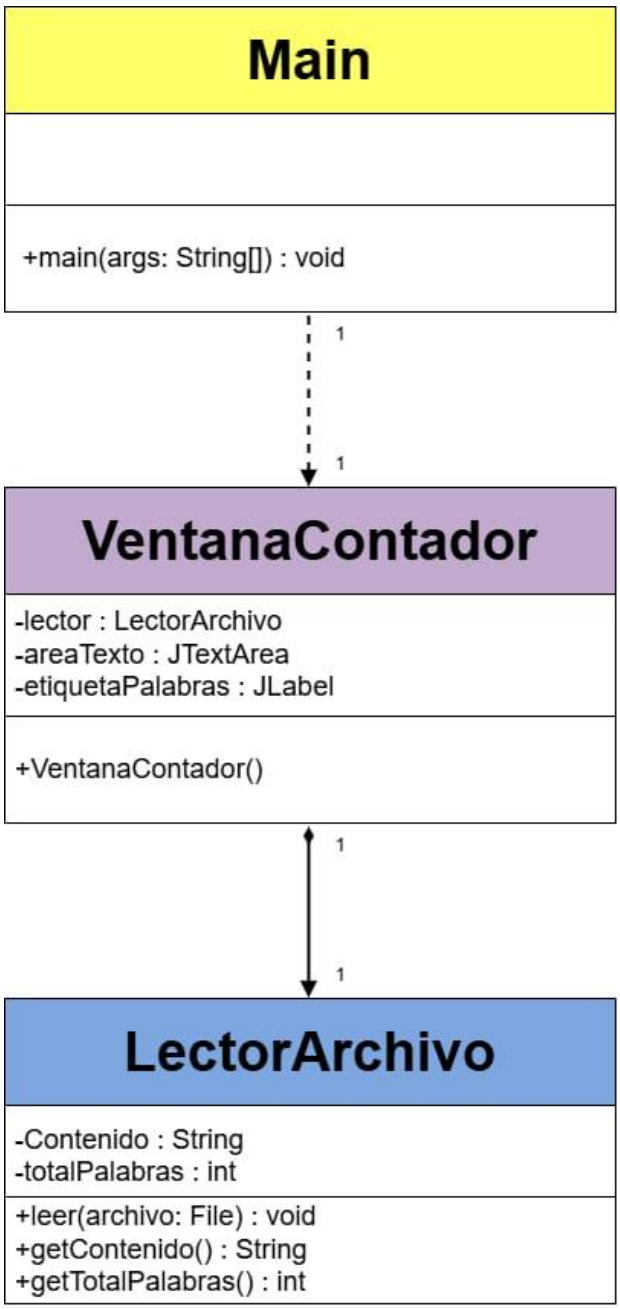


Figura 1. Diagrama de clases UML del sistema, mostrando la estructura estática y las relaciones de asociación entre las clases Main, VentanaContador y LectorArchivo.

Diagrama de Secuencia: Modela la dimensión temporal del flujo de ejecución. Ilustra cómo el usuario inicia la aplicación, el método main invoca al constructor de VentanaContador, la ventana inicializa el objeto LectorArchivo, configura las dimensiones del contenedor gráfico, ejecuta la lectura del archivo .txt, captura posibles excepciones de E/S, recupera el contenido textual y el total de palabras mediante métodos consultores, y finalmente actualiza los componentes gráficos antes de hacer visible la ventana al usuario.

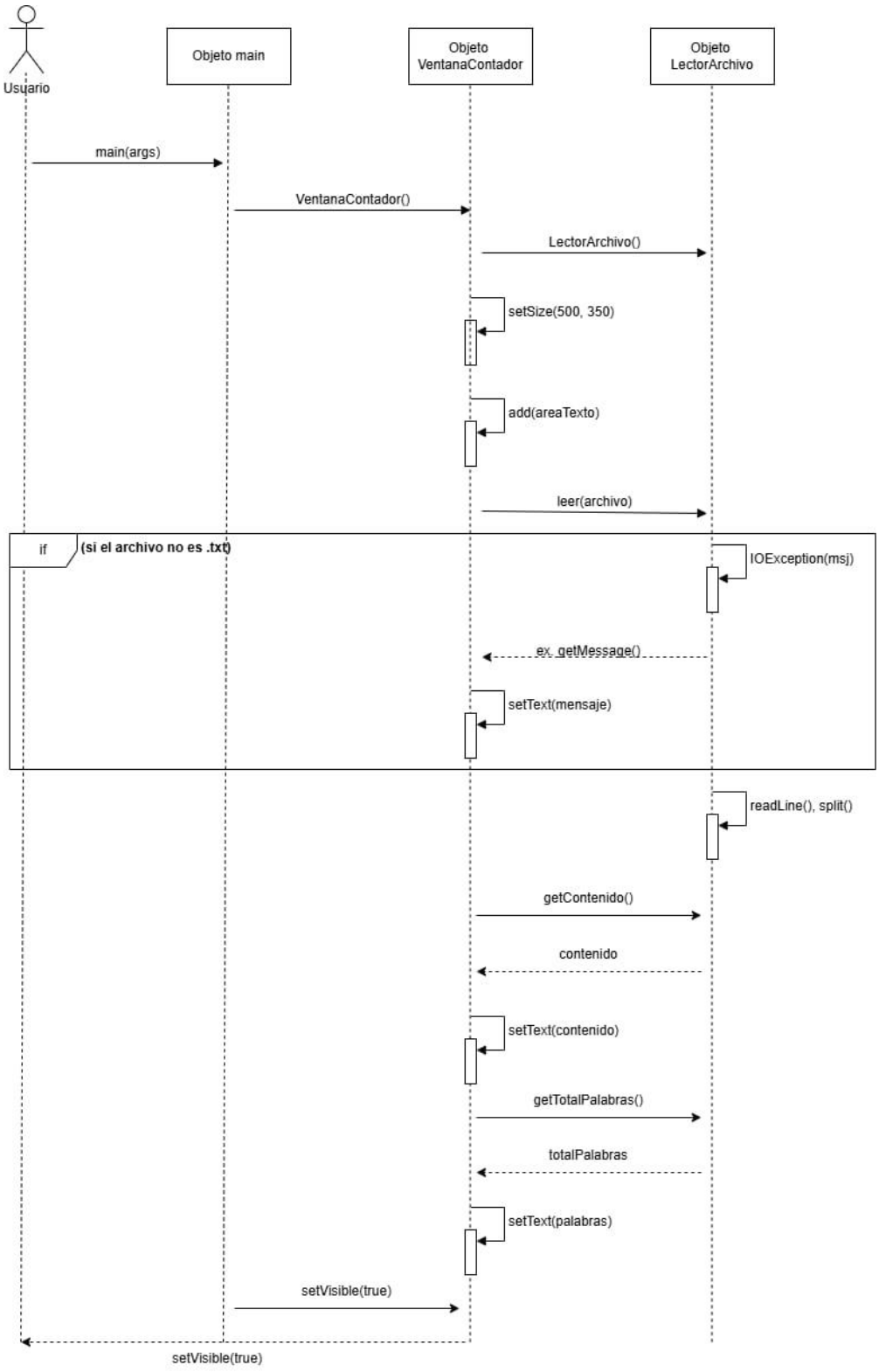


Figura 2. Diagrama de secuencia temporal que modela la interacción secuencial entre los objetos del sistema durante la apertura, lectura y despliegue gráfico del archivo de texto.

3.4. Pruebas de Escritorio

Para verificar la correcta integración entre el manejo de archivos y la interfaz gráfica, se ejecutó una metodología de pruebas basada en los siguientes escenarios:

Prueba 1: Lectura Nominal de Archivo .txt y Recuento de Palabras

Entrada: Archivo externo de texto plano con extensión .txt conteniendo párrafos descriptivos sobre la práctica.

Flujo: El sistema abre el flujo de lectura, procesa las líneas del documento, calcula el número exacto de palabras y transfiere los datos a los componentes visuales de la ventana.

Salida esperada: Despliegue íntegro del texto del archivo dentro del cuadro de texto gráfico y visualización exacta del recuento total en la etiqueta inferior (Total de palabras: 53), validando simultáneamente el Ejercicio 1 y el Ejercicio 2.

Prueba 2: Validación ante Archivos con Formato No Compatible

Entrada: Intento de lectura de un archivo con extensión distinta a .txt.

Flujo: El sistema evalúa la extensión del archivo, intercepta la excepción generada y evita la propagación de un error crítico.

Salida esperada: Supresión del cierre abrupto del programa y despliegue del mensaje de error controlado en la interfaz gráfica.

4. RESULTADOS

A continuación se presentan las descripciones técnicas correspondientes a las evidencias gráficas de los diagramas de diseño y la ejecución final de la aplicación:



Figura 3. Captura de ejecución de la interfaz gráfica de la aplicación, mostrando la correcta lectura del contenido de un archivo .txt desplegado en el cuadro de texto principal y el cálculo automatizado reflejado en la parte inferior (Total de palabras: 53).

5. CONCLUSIONES

El desarrollo de la práctica permitió consolidar los conceptos teóricos y prácticos relativos al intercambio de información entre fuentes de almacenamiento externo y las aplicaciones orientadas a objetos. La implementación de la clase encargada de la lectura de archivos evidenció la importancia de abstraer las operaciones de entrada y salida mediante flujos estructurados, garantizando que el acceso a discos se realice de manera limpia y controlada bajo el manejo riguroso de excepciones. Asimismo, la integración de los datos procesados con componentes de interfaz gráfica de usuario demostró la efectividad de los principios de diseño modular al separar la lógica de negocio y de archivos de los elementos visuales de presentación. La resolución de los ejercicios asignados reafirmó que el dominio de las estructuras de manipulación de cadenas y la gestión de componentes de texto en ventanas proporciona a los ingenieros en computación las bases metodológicas necesarias para construir aplicaciones de escritorio robustas, funcionales y preparadas para interactuar eficazmente con su entorno operativo.

6. REFERENCIAS

[1] Oracle Corporation, "I/O Streams," *The Java Tutorials: Essential Classes*. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/essential/io/streams.html

[2] Baeldung, "Reading a File in Java," *Baeldung Computer Science & Java Core Tutorials*. [En línea]. Disponible en: https://www.baeldung.com/java-read-file

[3] GeeksforGeeks, "File Handling in Java," *GeeksforGeeks Computer Science Portal*. [En línea]. Disponible en: https://www.geeksforgeeks.org/file-handling-in-java/

[4] Oracle Corporation, "Creating a GUI With Swing," *The Java Tutorials: Graphical User Interfaces*. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/uiswing/index.html

[5] Oracle Corporation, "TextArea (JavaFX 8 API Documentation)," *Oracle Documentation*, [En línea]. Disponible en:
https://docs.oracle.com/javase/8/javafx/api/javafx/scene/control/TextArea.html

[6] Facultad de Ingeniería, Universidad Nacional Autónoma de México (FI UNAM), "Manual de Prácticas del Laboratorio de Programación Orientada a Objetos (Lineamientos LCP02)," Ciudad Universitaria, México: UNAM, 2026. [En línea]. Disponible en: http://lcp02.fi-b.unam.mx/